

Object-Oriented Programming (OOP) Assignment

Four Fundamental Pillars • 16 Unique Implementation Programs in C++

Assignment Overview: This document demonstrates complete, production-ready, and unique C++ programs illustrating each of the 4 OOP pillars: *Encapsulation*, *Inheritance*, *Polymorphism*, and *Abstraction*. Each pillar contains 4 unique real-world programs designed with proper data types, access specifiers, and standards exceeding the minimum 10-line requirement.

Pillar 1: Encapsulation (Data Hiding & Protection)

Program 1.1: Smart Home Thermostat (Range & Limit Protection)

C++ • Encapsulation

```
#include <iostream>
using namespace std;

class SmartThermostat {
private:
    int targetTemp;
    bool ecoMode;

public:
    SmartThermostat() {
        targetTemp = 24;
        ecoMode = false;
    }

    void setTemperature(int temp) {
        if (temp >= 16 && temp <= 30) {
            targetTemp = temp;
            cout << "Target temperature set to " << targetTemp << " C." << endl;
        } else {
            cout << "Temperature out of safe operating limits (16-30 C)!" << endl;
        }
    }

    int getTemperature() {
        return targetTemp;
    }
};

int main() {
    SmartThermostat ac;
    ac.setTemperature(22);
    ac.setTemperature(10);
    cout << "Current AC setting: " << ac.getTemperature() << " C" << endl;
    return 0;
}
```

```
#include <iostream>
#include <string>
using namespace std;

class UserProfile {
private:
    string username;
    string bioText;

public:
    UserProfile(string user) {
        username = user;
        bioText = "Default bio";
    }

    void updateBio(string newBio) {
        if (newBio.length() <= 80) {
            bioText = newBio;
            cout << "Bio updated successfully for @" << username << endl;
        } else {
            cout << "Error: Bio exceeds the 80 character limit!" << endl;
        }
    }

    string getBio() {
        return bioText;
    }
};

int main() {
    UserProfile user("tech_nomad");
    user.updateBio("Full-stack developer building scalable web systems and cloud apps.");
    cout << "Active Bio: " << user.getBio() << endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;

class ATMCARDSecurity {
private:
    int pinCode;
    int failedAttempts;
    bool isCardLocked;

public:
    ATMCARDSecurity(int initialPin) {
        pinCode = initialPin;
        failedAttempts = 0;
        isCardLocked = false;
    }

    bool verifyPin(int enteredPin) {
        if (isCardLocked) {
            cout << "Card is blocked! Visit your branch." << endl;
            return false;
        }
        if (enteredPin == pinCode) {
            failedAttempts = 0;
            cout << "Access Granted." << endl;
            return true;
        } else {
            failedAttempts++;
            cout << "Wrong PIN! Attempts: " << failedAttempts << "/" << 3 << endl;
            if (failedAttempts >= 3) {
                isCardLocked = true;
                cout << "Card locked due to multiple incorrect attempts." << endl;
            }
            return false;
        }
    }
};

int main() {
    ATMCARDSecurity card(4821);
    card.verifyPin(1111);
    card.verifyPin(4821);
    return 0;
}
```

```
#include <iostream>
using namespace std;

class VehicleFuelTank {
private:
    double fuellitres;
    const double maxCapacity = 50.0;

public:
    VehicleFuelTank() : fuellitres(5.0) {}

    void addFuel(double litres) {
        if (litres <= 0) {
            cout << "Invalid fuel quantity." << endl;
            return;
        }
        if (fuellitres + litres <= maxCapacity) {
            fuellitres += litres;
            cout << "Added " << litres << "L fuel. Current fuel: " << fuellitres << "L" << endl;
        } else {
            cout << "Overflow warning! Tank capacity is only " << maxCapacity << "L." << endl;
        }
    }

    double getFuelLevel() {
        return fuellitres;
    }
};

int main() {
    VehicleFuelTank tank;
    tank.addFuel(20.0);
    tank.addFuel(40.0);
    return 0;
}
```

Pillar 2: Inheritance (Code Reusability & Hierarchy)

Program 2.1: Single Inheritance (Hospital Patient → ICU Patient)

C++ • Inheritance

```
#include <iostream>
#include <string>
using namespace std;

class Patient {
protected:
    string patientName;
    int wardNumber;

public:
    void registerPatient(string name, int ward) {
        patientName = name;
        wardNumber = ward;
    }
};

class ICUPatient : public Patient {
private:
    int ventilatorNumber;

public:
    void assignVentilator(int ventId) {
        ventilatorNumber = ventId;
    }

    void displayICUStatus() {
        cout << "ICU Patient: " << patientName << " | Ward: " << wardNumber
            << " | Ventilator Unit: #" << ventilatorNumber << endl;
    }
};

int main() {
    ICUPatient p;
    p.registerPatient("Hamza", 4);
    p.assignVentilator(12);
    p.displayICUStatus();
    return 0;
}
```

```
#include <iostream>
using namespace std;

class Spacecraft {
public:
    void initializeSystems() {
        cout << "[1] Telemetry and life-support active." << endl;
    }
};

class RocketBooster : public Spacecraft {
public:
    void igniteEngines() {
        cout << "[2] Rocket boosters ignited: Orbit achieved." << endl;
    }
};

class LunarLander : public RocketBooster {
public:
    void touchSurface() {
        cout << "[3] Landing gear deployed: Touched down on Lunar surface." << endl;
    }
};

int main() {
    LunarLander apollo;
    apollo.initializeSystems();
    apollo.igniteEngines();
    apollo.touchSurface();
    return 0;
}
```

```
#include <iostream>
#include <string>
using namespace std;

class LibraryResource {
protected:
    string title;

public:
    void setResourceTitle(string t) {
        title = t;
    }
};

class PrintedBook : public LibraryResource {
public:
    void issuePhysicalCopy(int rackNum) {
        cout << "Physical Book " << title << " issued from Shelf " << rackNum << endl;
    }
};

class AudioBook : public LibraryResource {
public:
    void streamAudio(string bitrate) {
        cout << "Streaming AudioBook " << title << " at quality: " << bitrate << endl;
    }
};

int main() {
    PrintedBook b;
    b.setResourceTitle("Clean Architecture");
    b.issuePhysicalCopy(7);

    AudioBook a;
    a.setResourceTitle("Deep Work");
    a.streamAudio("320 kbps");
    return 0;
}
```

```
#include <iostream>
using namespace std;

class HeartRateMonitor {
public:
    void trackPulse() {
        cout << "Heart Rate: 72 BPM (Resting)" << endl;
    }
};

class NavigationUnit {
public:
    void getCoordinates() {
        cout << "GPS: 32.0836 N, 72.6711 E" << endl;
    }
};

class FitnessSmartWatch : public HeartRateMonitor, public NavigationUnit {
public:
    void startWorkoutSession() {
        cout << "--- Workout Tracking Initialized ---" << endl;
        trackPulse();
        getCoordinates();
    }
};

int main() {
    FitnessSmartWatch watch;
    watch.startWorkoutSession();
    return 0;
}
```

Pillar 3: Polymorphism (Runtime & Compile-Time Behaviors)

Program 3.1: Runtime Polymorphism (Virtual Methods & Document Exporter)

C++ • Polymorphism

```
#include <iostream>
using namespace std;

class DocumentExporter {
public:
    virtual void exportData() {
        cout << "Exporting generic plaintext format." << endl;
    }
    virtual ~DocumentExporter() {}
};

class PDFExporter : public DocumentExporter {
public:
    void exportData() override {
        cout << "Exporting file as formatted Adobe PDF with vector fonts." << endl;
    }
};

class CSVExporter : public DocumentExporter {
public:
    void exportData() override {
        cout << "Exporting tabular records as comma-separated CSV." << endl;
    }
};

int main() {
    DocumentExporter* exporter = new PDFExporter();
    exporter->exportData();
    delete exporter;

    exporter = new CSVExporter();
    exporter->exportData();
    delete exporter;
    return 0;
}
```

```
#include <iostream>
using namespace std;

class VolumeCalculator {
public:
    double computeVolume(double side) {
        return side * side * side;
    }

    double computeVolume(double radius, double height) {
        return 3.14159 * radius * radius * height;
    }

    double computeVolume(double length, double width, double height) {
        return length * width * height;
    }
};

int main() {
    VolumeCalculator vc;
    cout << "Cube (side 4): " << vc.computeVolume(4.0) << endl;
    cout << "Cylinder (r=3, h=7): " << vc.computeVolume(3.0, 7.0) << endl;
    cout << "Box (l=5, w=2, h=4): " << vc.computeVolume(5.0, 2.0, 4.0) << endl;
    return 0;
}
```

```
#include <iostream>
#include <string>
using namespace std;

class NotificationService {
public:
    virtual void sendAlert(string message) {
        cout << "Broadcasting standard notice: " << message << endl;
    }
    virtual ~NotificationService() {}
};

class SMSAlert : public NotificationService {
public:
    void sendAlert(string message) override {
        cout << "[SMS Gateway] Sending 160-char SMS: " << message << endl;
    }
};

class PushNotification : public NotificationService {
public:
    void sendAlert(string message) override {
        cout << "[FCM Cloud] Pushing app notification banner: " << message << endl;
    }
};

int main() {
    NotificationService* notifier = new SMSAlert();
    notifier->sendAlert("OTP verification: 9382");
    delete notifier;

    notifier = new PushNotification();
    notifier->sendAlert("New message alert received.");
    delete notifier;
    return 0;
}
```

```
#include <iostream>
using namespace std;

class Vector2D {
private:
    double x;
    double y;

public:
    Vector2D(double xVal = 0.0, double yVal = 0.0) : x(xVal), y(yVal) {}

    Vector2D operator+(const Vector2D& other) {
        return Vector2D(this->x + other.x, this->y + other.y);
    }

    void printVector() {
        cout << "Vector Position: (" << x << ", " << y << ")" << endl;
    }
};

int main() {
    Vector2D force1(12.5, 4.0);
    Vector2D force2(3.5, 8.5);
    Vector2D netForce = force1 + force2;

    cout << "Combined resultant force:" << endl;
    netForce.printVector();
    return 0;
}
```

Pillar 4: Abstraction (Interfaces & Pure Virtual Functions)

Program 4.1: Cloud Storage Service Interface

C++ • Abstraction

```
#include <iostream>
#include <string>
using namespace std;

class CloudStorage {
public:
    virtual void uploadFile(string fileName) = 0;
    virtual void downloadFile(string fileId) = 0;
    virtual ~CloudStorage() {}
};

class AmazonS3Storage : public CloudStorage {
public:
    void uploadFile(string fileName) override {
        cout << "Encrypting and streaming " << fileName << " to AWS S3 bucket." << endl;
    }
    void downloadFile(string fileId) override {
        cout << "Retrieving blob object [" << fileId << "] via AWS presigned URL." << endl;
    }
};

int main() {
    CloudStorage* storage = new AmazonS3Storage();
    storage->uploadFile("user_avatar.png");
    storage->downloadFile("obj_98231");
    delete storage;
    return 0;
}
```

Program 4.2: Digital Audio Codec Stream Decoder

C++ • Abstraction

```
#include <iostream>
using namespace std;

class AudioDecoder {
public:
    virtual void decodePacketStream() = 0;
    virtual ~AudioDecoder() {}
};

class FLACDecoder : public AudioDecoder {
public:
    void decodePacketStream() override {
        cout << "Decompressing lossless 24-bit FLAC audio stream..." << endl;
        cout << "Outputting PCM raw sample buffers to DAC." << endl;
    }
};

int main() {
    AudioDecoder* playerDecoder = new FLACDecoder();
    playerDecoder->decodePacketStream();
    delete playerDecoder;
    return 0;
}
```

Program 4.3: Digital Signature & Cryptographic Signer

C++ • Abstraction

```
#include <iostream>
#include <string>
using namespace std;

class CryptoSigner {
public:
    virtual void generateSignature(string payload) = 0;
    virtual ~CryptoSigner() {}
};

class RSASigner : public CryptoSigner {
public:
    void generateSignature(string payload) override {
        cout << "Hashing input: " << payload << endl;
        cout << "Signing digest with RSA 4096-bit private key." << endl;
    }
};

int main() {
    CryptoSigner* signer = new RSASigner();
    signer->generateSignature("TxAmount=5000;Receiver=Umar");
    delete signer;
    return 0;
}
```

Program 4.4: Industrial Robotic Joint Controller

C++ • Abstraction

```
#include <iostream>
using namespace std;

class RoboticJoint {
public:
    virtual void calibrateSensors() = 0;
    virtual void rotateToAngle(double degrees) = 0;
    virtual ~RoboticJoint() {}
};

class ServoArmJoint : public RoboticJoint {
public:
    void calibrateSensors() override {
        cout << "Zeroing servo encoder home position." << endl;
    }
    void rotateToAngle(double degrees) override {
        cout << "Actuating stepper motor to " << degrees << " degrees rotation." << endl;
    }
};

int main() {
    RoboticJoint* joint = new ServoArmJoint();
    joint->calibrateSensors();
    joint->rotateToAngle(90.0);
    delete joint;
    return 0;
}
```